CHAPTER 2

Data Types, Operators, and Expressions

Abstract: Now that you have been familiarised with the installation and basics of Python programming, it's time to dig in a little more and understand the different data types that are available along with the operators and expressions that make the programming more user-friendly. In this chapter, we will learn about data type categorization and the operators present. Here you will learn the following nuances:

1. Data types supported by the Python.
2. Use of variables to store and access the data.
3. Operators do mathematical works and logical functions.
4. Variety of expressions to serve the range of applications.

Keywords: Associativity, Literals, Operators, Precedence.

INTRODUCTION

In computer programming, a data type is a classification of the many kinds of information that may be saved in a variable. Since Python is a dynamically typed language, we do not need to specify the variable type when we declare it.

The value is bound to its type in a way that the interpreter does not explicitly specify—for example, **X=100**. We did not declare the type of the variable **X**, yet it now holds the integer value 100. In Python, the integers are by default treated to be of integer data type. To check the type of the variable, we use **type()** function available to us in Python, and it returns the type of the variable that was handed in. The following example illustrates how to specify the values for a variable and to check their types.

Example:

```
1.    x=100
2.    y="Python Programming"
3.    z= 12.59
4.    print (type(x))
5.    print (type(y))
6.    print (type(z))
```

Krishna Kumar Mohbey & Malika Acharya

Output:

```
<class 'int'>
<class 'str'>
<class 'float'>
```

Python has a wide variety of standard data types, each of which can have a unique storage mechanism defined for it. The data types defined in Python are listed below:

1. Numbers
2. Sequence
3. Dictionary
4. Boolean
5. Set

NUMBER

The number is responsible for storing numerical values. A Python Numbers data type is responsible for storing all integer, float, and complex values. To check the data type of a variable, Python has a function called **type()**. It also has the **isinstance()** function that checks whether or not an object belongs to a specific class. When a number is assigned to a variable in Python, Number objects are created automatically. A number data type can store int, float, and complex type values.

- The integer value can be any length. There is no limitation on the length of an integer in Python.
- Float is used to store floating-point numbers which is accurate up to 15 decimal points.
- A complex number is specified in the form of **a + ib**, where **a** and **b** stand for the real and imaginary components of the number, respectively.

Example:

```
a = 10
print ("The type of a is", type(a))

b = 10.15
print ("The type of b is", type(b))

c = 2+4j
print ("The type of c is", type(c))
print ("c is a complex number", isinstance(2+4j,complex))
```

Output:

```
The type of a is <class 'int'>

The type of b is <class 'float'>

The type of c is <class 'complex'>

c is a complex number True
```

Sequence

The language treats Python's string, list, and tuple datatypes as sequence types. The character sequence enclosed in quote marks is an example of the string, which may be defined as that sequence. When defining a string, Python allows us to use single quotes, double quotes, or triple quotations. As Python includes predefined functions and operators that may be used to execute various actions on strings, managing strings in Python is a simple process. The *"Hello World!"* is the result of the operation *"Hello"* + *"World!",* which uses the plus (+) operator to concatenate the two strings. Similarly to repeat the strings we can use the repetition operator (*). For example "India" *7 outputs IndiaIndiaIndiaIndiaIndiaIndiaIndia '.

Example:

```
S1= "Hello World"
print (S1)
```

Output:

```
Hello World
```

Lists in Python function is similar to arrays in C. The list, however, may include information of varying formats. The list's contents are surrounded in square brackets [] and separated by commas (,). We may selectively get the list's contents by using slice [:] operators. The list may be manipulated using the same concatenation (+) and repetition (*) operators as strings.

Example:

```
1.  L1 = [10, "Hello", "World!", 20, 30]
2.  L2 = [1, "Hello", 2.0]
3.  #Checking the type of given list
4.  print (type(L1))
5.  print (type(L2))
6.  #Printing the list
7.  print (L1)
8.  print (L1[3:])
9.  print (L2[0:1])
10.
11.  # List Concatenation using + operator
12.  print (L1 + L2)
13.
14.  # List repetition using * operator
15.  print (L2 * 4)
```

Output:

```
<class 'list'>

<class 'list'>

[10, 'Hello', 'World!', 20, 30]

[20, 30]

[1]

[10, 'Hello', 'World!', 20, 30, 1, 'Hello', 2.0]

[1, 'Hello', 2.0, 1, 'Hello', 2.0, 1, 'Hello', 2.0, 1, 'Hello', 2.0]
```

A tuple is very much like a list in many ways. Tuples, much like lists, are collections of things that can be of any data type. We use parenthesis to specify the components of a tuple and different elements of the tuple are separated by commas (,). Because the size and value of the elements included in a tuple cannot be changed, the tuple data structure is said to be read-only.

Example:

```
1.  T1 = [10, "Hello", "World!", 20, 30]
2.  T2 = [1, "Hello", 2.0]
3.
4.  # Checking type of tuples
5.  print (type(T1))
6.  print (type(T2))
7.  #Printing the tuple
8.  print (T1)
9.
10. # Tuple slicing
11. print (T1[2:])
12. print (T1[0:1])
13.
14. # Tuple concatenation using + operator
15. print (T1 + T2)
16.
17. # Tuple repetition using * operator
18. print (T1 * 2)
```

Output:

```
<class 'list'>

<class 'list'>

[10, 'Hello', 'World!', 20, 30]

['World!', 20, 30]

[10]

[10, 'Hello', 'World!', 20, 30, 1, 'Hello', 2.0]

[10, 'Hello', 'World!', 20, 30, 10, 'Hello', 'World!', 20, 30]
```

DICTIONARY

A dictionary is a collection of key-value pairs that are not arranged in any order. It functions similarly to an associative array or a hash table, in which the individual keys each contain a distinct value. The value of a key can be any arbitrary Python object, although the key itself can store any primitive data type. The entries in the dictionary are delineated from one another using commas (,), and they are encapsulated in curly braces ().

Example:

```
Dic = {101:'Amit', 102:'Sumit', 103:'Rahul', 104:'James'}
 # Printing dictionary
print (Dic)
print("Your first record is : "+Dic[101])
print("Your fourth record is:  "+ Dic[104])
# Printing keys and values
print (Dic.keys())
print (Dic.values())
```

Output:

```
{101: 'Amit', 102: 'Sumit', 103: 'Rahul', 104: 'James'}
Your first record is: Amit
Your fourth record is: James
dict_keys([101, 102, 103, 104])
dict_values(['Amit', 'Sumit', 'Rahul', 'James'])
```

BOOLEAN

The **True** and **False** values are predefined using the Boolean data type. The supplied values are compared to these to check whether the assertion is true or false. The bool class is used to indicate what it is. The value **'T'** or any other value that is not zero can represent true, whereas the value **'0'** or the letter **'F'** can be used to represent false.

Example:

```
x = bool(1)
#display x:
print (x)
#display the data type of x:
print (type(x))
```

Output:

```
True
<class 'bool'>
```

SET

The data type known as Set in Python is a collection that is not sorted. It is possible to edit it after it has been created, it may be repeated, and it has components that are not found anywhere else. The order in which the items are presented in a set is indeterminate, hence the return sequence may be in an alternative order. The built-in method **set()** is used to specify the set data type**;** alternatively, a series of items may be given between the curly braces and delimited by commas to build the set. It is capable of holding a wide variety of values.

Example:

```
1.  emp_set = set()
2.  set_data = {'Walter', 74, 'Book'}
3.  #Display values of the set
4.  print (set  data)
5.  #Inserting data into the  set
6.  set_data.add(1282)
7.  print (set_data)
8.  #Delete a record  from the set
9.  set_data.remove(74)
10. print (set_data)
```

Output:

```
{'Book', 'Walter', 74}

{'Book', 'Walter', 1282, 74}

{'Book',  'Walter', 1282}
```

TYPE CONVERSION

It converts the value from one data type to another. The data type may be an integer, string, float, *etc*. There are two types of conversion: *Implicit Type Conversion* and *Explicit Type Conversion.*

Implicit Type Conversion

In this conversion, Python automatically converts one data type to another data type. A user is not involved in this type of conversion. Let us take an example of implicit type conversion.

Example:

```
num_int = 234
num_float = 23.4
num_new= num_int + num_float
print ("datatype of num_int:",type(num_int))
print ("datatype of num_flo:",type(num_float))
print ("Value of num_new:",num_new)
print ("datatype of num_new:",type(num_new))
```

Output:

```
datatype of num_int: <class 'int'>
datatype of num_flo: <class 'float'>
Value of num_new: 257.4
datatype of num_new: <class 'float'>
```

Explicit Type Conversion

In this conversion, users convert the data type of an object to the required data type. It uses some predefined functions like *int(), float(), str()*, *etc*., to perform explicit type conversion. Moreover, it is also called typecasting because the user casts (changes) the objects' data type. For typecasting, the variable for which the type conversion is required is specified along with the requisite data type. There is the following syntax of explicit type casting.

```
<required_datatype>(expression)
```

Example:

```
x = 345
y = "867"
print ("x has the type :",type(x))
print ("y has the type :",type(y))

y = int(y)
print ("After Type Casting y is :",type(y))
sum = x + y
print ("Sum :", sum)
print ("Sum has the data type:",type(sum))
```

Output:

```
x has the type : <class 'int'>
y has the type : <class 'str'>
After Type Casting y is : <class 'int'>
Sum :1212
Sum has the data type : <class 'int'>
```

OPERATORS

In Python, the idea of carrying out anything from a computational perspective is called an operator. In this context, we use two different terminology forms: operators and operands. An operand is a value being operated on by an operator, and an operator is a symbol being executed with the assistance of the operands. It is a special kind of symbol that carries out an operation on its operands and returns the outcome of that action. Take statements **10 + 20** as an example to help you better comprehend the concept. The numbers 10 and 20 make up the operands, while the plus **(+)** sign is the operator. Therefore, the evaluation of this example will consist of a straightforward arithmetic expression, and the outcome will be the addition of two numbers. The various operators available in Python are enlisted below:

1. Arithmetic Operators
2. Relational Operators
3. Assignment Operator
4. Unary Operators
5. Bitwise Operators
6. Logical Operators
7. Membership Operators
8. Identity Operators

Arithmetic Operators

It performs the basic mathematical operation on the numeric operands. The arithmetic operators return the result, and the outcome depends on the type of operands you use in programming. Arithmetic operators are listed below:

Addition (+) Operator

The symbol + is used while using this operator. It uses two operands and adds the two values. For example, a=1, b=2, then a + b returns 3.

Subtraction (-) Operator

The symbol – (minus) is used while using this operator. It uses two operands, and this operator subtracts the value and returns the result. For example, a=4, b=2, then a - b returns 2.

Multiplication (*) Operator

The symbol * is used while using this operator. It operates between two operands, and this operator multiplies these values and returns the result. For example, a=4, b=2, then a*b returns 8.

Division (/) Operator

The symbol (/) uses while using this operator. It uses two operands; this operator divides the value and returns the result. For example, a=4, b=2, then a / b returns 2.

Modulus (%) Operator

The symbol % is used while using this operator. It uses two operands, and this operator returns the remainder. For example, a=4, b=2, then a % b returns 0.

Exponent (**) Operator

The symbol ** is used while using this operator. It operates between two operands. It performs exponential (power) calculations on operators.

For example, a=4, b=2, then a ** b means $(4)^2$. The result will be 8.

Floor Division (//) Operator

Floor division is like regular division but returns the highest possible integer. This integer is less than or equal to the outcome of normal division.

For example, 7//2 = 3 and 7.0//2.0 = 3.0, -11//3 = -4, -11.0//3 = -4.0

Example:

```
a=4
b=2
f=a/b        #Division (/)
print (f)
g=a%b        # Modulus (%)
print (g)
h=a**b       #Exponent (**)
print (h)
i=a//b       #Floor Division (//)
print (i)
```

Output:

```
2.0
0
16
2
```

Relational Operators

They are used to compare the variables and output the relation. It is also called a comparison operator. This type of operator returns a Boolean value (*True/False*). There are various types of relational operators.

Greater than (>) Operator

It returns true if the left operand is greater than the right. Example, (a>b).

Greater than or equal to (>=) Operator

It returns true if the left operand is greater than or equal to the right. Example, (a>=b).

Less than (<) Operator

It returns true if the left operand is less than the right. Example, (a<b).

Less than or equal to (<=) Operator

It returns true if the left operand is less than or equal to the right. Example, (a<=b).

Equal to (==) Operator

It returns true if both operands are equal. Example, (a==b).

Not equal to (! =) Operator

It returns true if operands are not equal. Example, (a != b).

Assignment Operator

It assigns the values for a variable. For example, to assign a value of 50 to the variable say **a** simple command is: **a = 50**. The assignment operator used in conjunction with other operators serves different purposes, such as **x += 10**, adds 10 to the variable, and later assigs the same. It is equivalent to **x = x + 10**. It is also called a shortcut operator. There are various types of assignment and shortcut operators.

Assignment (=) Operator

This operator assigns values from right-side operands to left-side operands. For example, **z = x + y** assigns **x + y** into **z**.

Add AND (+=) Operator

This operator adds the right operand to the left operand and assigns the result to the left operand. For example, **z += x** is equivalent to **z = z + x**.

Subtract AND (-=) Operator

This operator subtracts the right operand from the left operand and assigns the result to the left operand. For example, **z -= x** equals **z = z – x**.

Multiply AND (=) Operator*

This operator multiplies the right operand with the left operand and assigns the result to the left operand. For example, **z *= x** is equivalent to **z = z * x**.

Divide AND (/=) Operator

This operator divides the left operand with the right operand and assigns the result to the left operand. For example, **z /= x** is equivalent to **z = z / x**.

Modulus AND (%=) Operator

This operator takes modulus using two operands and assigns the result to the left operand. For example, **z %= x** is equivalent to **z = z % x**.

*Exponent AND (**=) Operator*

This operator performs exponential (power) calculation on operators and assigns value to the left operand. For example, **z **= x** is equivalent to **z = z ** x**.

Floor Division (//=) Operator

This operator performs floor division on operators and assigns value to the left operand. For example, **z //= x** is equivalent to **z = z // x**.

Example:

```
x = 10
y = 20
#value modification
val= x + y
print ( " val : ", z)
val2 += x
print ( " val2 : ", z)
val3 *= x
print ( "val3: ", z)
val4/= x
print ( "val4 : ", z)
num= 2
num %= x
print ( "num:  ", z)
num **= x
print ( "num:  ", z)
num //= x
print ( "num: ", z)
```

Output:

```
val: 30

val2:40

val3:400

val4:40.0

num:2

num:1024

num:02
```

Unary Operators

It is used in Python and performed on a single operand. The two unary operators are: **unary (+)** and **unary (-).** Let us take an example for understanding unary operators.

Example:

```
1.    x = 1
2.    print ("Unary positive operator", +x)
3.    print ("Unary Negative operator", -x)
```

Output:

```
Unary positive operator 1

Unary Negative operator -1
```

Bitwise Operators

The Bitwise operator operates bit by bit. Suppose **x = 26** and **y = 18**; then the binary conversion of x and y is 11010, 10010. With their results, let us understand the various bitwise operations on **x** and **y**.

There are the following Bitwise operators:

Binary AND (&) Operator

This operator copies a bit of the result if it exists in both operands.

Binary OR (\|) Operator

This operator copies a bit if it exists in either operand.

Binary XOR (^) Operator

This operator copies the bit if it is set in one operand but not both.

Binary One's Complement (~) Operator

It is unary and has the effect of 'flipping' bits.

Binary Left Shift (<<) Operator

In this operator, the left operand's value is moved left by the number of bits specified by the right operand.

Binary Right Shift (>>) Operator

In this operator, the value of the left operand is moved right by the number of bits specified by the right operand. Let us take an example for understanding all bitwise operators.

Example:

```
x = 26
y = 18
z = 0
z = x & y,
print ( "Ans for '&': ", z)
z = x | y,
print ( " Ans for '|':", z)
z = x ^ y,
print ( " Ans for '^':", z)
z = ~x,
print ( " Ans for '~':", z)
z = x << 2,
print ( "Ans for '<<':", z)
z = x >> 2,
print ( " Ans for '>>':", z)
```

Output:

```
Ans for '&':  18
Ans for '|':  26
Ans for '^': 8
Ans for '~': -27
Ans for '<<":  104
Ans for '>>':, 6
```

Logical Operators

Logical operators are used to perform arithmetic and logical computations. These are the special symbols **(and, or, not)** for accomplishing the operations. These operators return True/False as a result.

Logical AND Operator

This operator returns true when both conditions become true.

Logical OR Operator

This operator returns true when two operands are non-zero.

Logical NOT Operator

This operator returns the reverse result, *i.e* the true condition becomes false and the false condition becomes true.

Example:

```
1.  x = 70
2.  print (x > 30 and x < 100)
3.  print (x > 30 or x < 40)
4.  print (not(x > 30 and x < 100))  # returns False because not is used to reverse the result
```

Output:

```
True

True

False
```

Membership Operators

A membership operator tests if a sequence is presented in an object. Here an object may be strings, lists, or tuples. There are two types of membership operators ***in*** and ***not***. Let us take an example of using these operators.

Operator (in)

The operator returns *True* if the variable sought is present in the sequence and *False* otherwise.

Operator (not)

The operator returns *True* if the variable sought is ***NOT*** present in the sequence and *False* otherwise.

Example:

```
x= 10
y = 20
list = [10, 12, 23, 44, 35 ],

if ( x in list ):
  print ( " x is available in the given list")
else:
  print ( " x is not available in the given list")

if ( y not in list ):
  print ( "y is not available in the given list")
else:
  print ( " y is available in the given list")

x = 2
if ( x in list ):
  print ( " x is available in the given list")
else:
   print ( " x is not available in the given list")
```

Output:

```
x is available in the given list

y is not available in the given list

x is not available in the given list
```

Identity Operators

This operator is used to compare the objects, not if they are equal, but if they are the same object, with the exact memory location. There are two types of identity operators, namely, ***is*** and ***is not***. Let us take an example of using these operators.

Operator (is)

This operator returns true if both variables are the same object.

Operator (is not)

This operator returns true if both variables are not the same object.

Example:

```
x = ["James", "Sam"]
y = ["James", "Sam"]
z = x
print (x is z) # returns True because z is the same object as x
print (x is y) # returns False because x is not the same object as y, even if they have the same content
print (x != y) # This comparison returns False because x is equal to y
```

Output:

```
True

False

False
```

Operators' Precedence and Associativity

Operator precedence means which operator should be executed first. It is based on the priority of an operator. If two or more operators have the same precedence, it will check an operator's associativity. Associativity means operating the value from left to right and right to left. Suppose an expression is **2*4+6**; first, **2*4** will be executed because * has higher precedence between the * and + operators.

Suppose an expression is **5/2*3**, then the operator's priority is the same, then it will check the associativity of an operator. Here associativity of / and * is left to right, thus 5/2 is evaluated first. Table **1** enumerates the various operators precedence-wise along with their associativity.

Table 1. Operators description and its associativity.

Operator	Description	Associativity
()	Parentheses	left-to-right
**	Exponent	right-to-left
* / %	Multiplication/division/modulus	left-to-right
+ –	Addition/subtraction	left-to-right
<< >>	Bitwise shift left, Bitwise shift right	left-to-right
< <= > >=	Relational	left-to-right
== !=	Relational is equal to/is not equal to	left-to-right

(Table 1) cont.....

Operator	Description	Associativity
is, is not **in, not in**	Identity Membership operators	left-to-right
&	Bitwise AND	left-to-right
^	Bitwise exclusive OR	left-to-right
|	Bitwise inclusive OR	left-to-right
not	Logical NOT	right-to-left
and	Logical AND	left-to-right
or	Logical OR	left-to-right
= **+= -=** ***= /=** **%= &=** ^= |= <<= >>=	Assignment Addition/subtraction assignment Multiplication/division assignment Modulus/bitwise AND assignment Bitwise exclusive/inclusive OR assignment Bitwise shift left/right assignment	right-to-left

EXPRESSIONS

Expression is a representation of value. Moreover, it combines values, variables, operators, and calls to functions. Expressions need to be evaluated. If you ask Python to print an expression, the interpreter evaluates it and displays the desired result. It is very different from the statements. A simple statement in Python is used for value assignment but it is not evaluated, whereas expressions are more logical content that is used in the evaluation. Python expressions only contain three things, Identifiers, Literals, and Operators.

Identifiers

It can be any name that is used to define a class, function, variable module, or object.

Literals

It can be string literals, byte literals, integer literals, floating-point literals, and imaginary literals.

Operators

Based on the token or symbol, it can evaluate an operation. There are various operators, as discussed above, and their corresponding tokens can be used to implement any expression.

STRING OPERATIONS

A string is an essential concept in Python because it can be used in coding in many places. The string can be created with either single or double quotation marks. Let us take an example of string creation.

S1= 'Python Programming.'

S2= "Python Programming."

Both are the way to create a string.

Accessing Values in Strings

An assignment is much easier than another programming language because character types are NOT supported in Python. Characters are considered strings of length one. If we want to take a subpart of a string, then we have to take a substring. The substring is determined by specifying the slice required using index/indices within the square brackets. A string can also update and reassign the value to another string. Let us take an example.

Example:

```
1.  wr1= 'Book for Beginners
2. wr2= 'Requires"
3. print ("wr1[0]: ", wr1[0])
4. print ("wr2[0:6]: ", wr2[0:6])
5. print ("Updated :- ", str2[:9] + 'Book')
```

Output:

```
wr1[0]:  B

wr2[0:5]:  Requir

Updated  :-  Requires Book
```

There are various types of special symbols used in the string:

- **Concatenation (+)** – It adds values on either side of the operator.
- **Repetition (*)**- It creates a new string, concatenating multiple copies of the same string.
- **Slice ([])** – It gives the character from the given index.
- **Range Slice ([:])**- It gives the characters from the given range.

Various types of formatted string operators decide the formatting of string. The different format specifiers in Python are specified by adding the '%' operator symbol before the required format. % symbol denotes the placeholder to insert the variable in the specified format.

%c character, %s string conversion, %d signed decimal integer, %u unsigned decimal integer, %o octal integer, %x hexadecimal integer (lowercase letters), %e exponential notation (with lowercase 'e'), %f floating point real number, %g short for %f and %e, %G the short for %f and %E.

Example:

```
1. print ("My name is %s and my student ID is %d." % (Bateron, 025))
```

Output:

```
My name is Bateron and my student ID  is 025.
```

Triple Quotes

It is used for writing a string in multiple lines. It can be written as triple quotes to three consecutive single or double quotes.

Example:

```
1.  Longstring = """ This is a long string example and that is made up of
2.  several lines and non-printable characters such as
3.  TAB ( \t ) and they will show up that way when displayed.
4.  NEWLINEs within the string, whether explicitly given like
5.  this within the brackets [ \n ], or just a NEWLINE within
6.  the variable assignment will also show up.
7.  """
8.  print Longstring
```

Output:

```
This is a long string example and that is made up of

several lines and non-printable characters such as

TAB (	) and they will show up that way when displayed.

NEWLINEs within the string, whether explicitly given like

this within the brackets [

], or just a NEWLINE within

the variable assignment will also show up.
```

Various types of built-in functions manipulate strings:

- *capitalize()*- It is used to put the first letter of the string in upper case.
- *center(width, fill char)*- It is used to pad the original string cantered to the total column width.
- *count(str, beg= 0,end=len(string))*-To count the number of times a specific word/sequence of words occurs within a string we use this function. It requires us to specify the beginning and the ending index of the string we need to search in.
- *decode(encoding='UTF-8',errors='strict')*- To return the decoded string we use this function.
- *encode(encoding='UTF-8',errors='strict')*- It returns the encoded string. By default, it raises a Value Error, but we can specify 'ignore' or 'replace'.
- *endswith(suffix, beg=0, end=len(string))*- It is used to check whether the given string/ substring ends with the specified suffix. It returns True if the condition is satisfied otherwise False.
- *expandtabs (tabsize=8)*: It is used to expand the scope of tabs to the specified tab size. By default, the tab size is 8.
- *find (str, beg=0 end=len(string))*- It is used to check if the string/ substring is present in the given string within the specified beginning and ending indices.
- *isalnum()* - It is used to check whether the string is composed of all alphanumeric characters or not. It outputs True if the string has one character and other alphanumeric characters, otherwise it returns False.
- *isalpha()*- It is used to determine whether the sting is composed of characters and alphabets.
- *isdigit()* – It returns True if the string contains only digits and False otherwise.
- *istitle()*- It returns True if the string is properly "title case" and False otherwise.
- *maketrans()*- It returns a translation table for the use in translate function.

- *max(str)*- It returns the max alphabetical character from the string.
- *min(str)*- It returns the min alphabetical character from the string str.
- *replace(old, new [, max])*- It is used to replace old string contents with the new or at most max occurrences specified within max.
- *rfind(str, beg=0,end=len(string))* – To search the string from backward we use this function.
- *rstrip()*- It strips off the trailing whitespaces.
- *split(str=", num=string.count(str))*- It is used to split the string based on the delimiter and return the substring that is split into at most *num* substrings.
- *splitlines(num=string.count('\n'))*- It splits a string at all (or num) NEWLINEs and returns a list of each line with NEWLINEs removed.
- *startswith(str, beg=0,end=len(string))*-It determines if a string or a substring of string (if starting index beg and ending index end are given) starts with substring str, returns true if so and false otherwise.
- *strip([chars])*-It is used to perform both lstrip() and rstrip() on string.
- *swapcase()*- It converts the case of all letters.
- *string.title()*- It returns the "titlecased" version of the string, that is, all words begin with uppercase, and the rest are lowercase.
- *upper()*- It is used to convert lowercase letters to uppercase.
- Let us take an example to understand all the above built-in functions.

Example:

```
#!/usr/bin/python
text = "This is a long string example and that is made up of several lines and character
       s"
print ("text.capitalize() : ", text.capitalize())
print ("text.center(40, 'a') : ", text.center(40, 'a'))
sub = "i";
print ("text.count(sub, 4, 40) : ", text.count(sub, 4, 40))
sub = "wow";
print ("text.count(sub) : ", text.count(sub))
text1 = "An operator, is a concept in Python ";
text2 = "Python";
print (text1.find(text2))
print (text1.find(text2, 10))
print (text1.find(text2, 40))
print (text1.index(text2))
text = "this2009";  # No space in this texting
print (text.isalnum())
text = "An operator, is a concept in Python";
print (text.isalnum())
text = "this";  # No space & digit in this texting
print (text.isalpha())
text = "An operator, is a concept in Python";
print (text.isalpha())
text = "123456";  # Only digit in this texting
print (text.isdigit())
text = "An operator, is a concept in Python";
print (text.isdigit())
text = "An operator, is a concept in Python";
print (text.islower())
text = "An operator, is a concept in Python";
print (text.islower())
text = u"this2009";
print (text.isnumeric())
text = u"23443434";
print (text.isnumeric())
text = "       ";
print (text.isspace())
text = "An operator, is a concept in Python";
print (text.isspace())

text = "Hello, welcome to the world of Python";
print (text.istitle())
print (text.isupper())

print (text.isupper())
s = "-";
seq = ("a", "b", "c"); # This is sequence of textings.
print (s.join( seq ))
```

Output:

```
text.capitalize() :  This is a long string example and that is made up of several lines and characters

text.center(40, 'a') :  This is a long string example and that is made up of several lines and characters

text.count(sub, 4, 40) :  3

text.count(sub) :  0

29

29

-1

29

True

False

True

False

True

False

False

False

False

True

True

False

False

False

False

a-b-c
```

CONCLUDING REMARKS

In this chapter, we covered a variety of different data types as well as operations. In addition, we spoke about the precedence of operators and their associativity when evaluating expressions—an explanation of the many principles involved in producing string operations, lists, and dictionaries. The next topic that will be covered is control flow statements, which are utilized in Python. This will be covered in the following chapter.